Compiler2026-RISCV

编译器设计说明

SysY2022 → RV64GC

前端 · 多级中间表示 · 优化流水线 · MIR 后端

2026 年 8 月

目录

1. 项目定位 3

2. 对外接口与两条编译通道 3

3. 总体架构 3

4. 源码组织 4

5. 前端设计 5

6. 功能通道 5

7. AST/HIR 高层变换 5

8. CoreIR 数据模型 6

9. 分析基础设施 6

10. CoreIR 优化流水线 6

11. 优化正确性原则 7

12. strict 与 research 隔离 8

13. MIR 与 RISC-V 后端 8

14. ABI、数据布局与汇编输出 8

15. SysY 运行时边界 9

16. 构建与使用 9

17. 测试与证据体系 10

18. 新增 pass 的接入流程 10

1. 项目定位

Compiler2026-RISCV 是使用 C++17 实现的 SysY2022 编译器。编译器读取 SysY 源程序，生成面向 RISC-V 64 位平台的汇编代码；生成结果可由比赛 平台提供的 SysY 运行时库链接，并在 RV64GC Linux 环境中执行。

项目覆盖一条完整编译链：词法分析、语法分析、抽象语法树、控制流与 数据流分析、多级中间表示优化、机器指令降低、寄存器分配、栈帧布局和 汇编输出。设计目标包括：

- 正确实现 SysY2022 语言与比赛规定的命令行接口；
- 将功能编译与性能编译的职责清晰分开；
- 以 AST、SSA、CFG、支配、循环、alias、effect 和目标成本等通用事实

驱动优化；

- 对无法证明等价的候选保持原程序，不依赖用例名称或固定输入；
- 生成符合 RISC-V ABI、可在较大地址空间使用的 RV64GC 汇编。

2. 对外接口与两条编译通道

比赛评测使用以下两个公开接口：

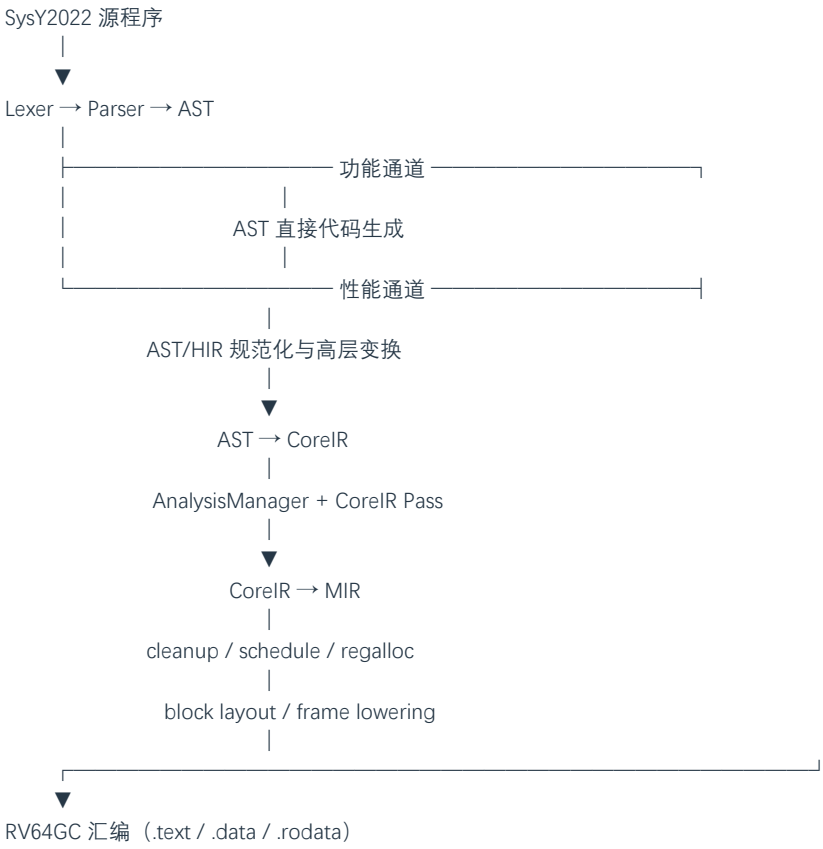
```
compiler -S -o testcase.s testcase.sy
compiler -S -o testcase.s testcase.sy -O1
```

两条命令共享同一套 lexer、parser 和 AST，但后续路径不同。

通道	入口	主要目标	后端
功能通道	不带 -O1	以较短路径稳定生成正确汇编	AST 直接代码生成器
性能通道	带 -O1	执行多级分析、优化和目标相关处理	CoreIR → MIR 后端

功能通道不是性能通道的“关闭全部 pass”版本，而是一条独立、直接的代码 生成路径。性能通道统一进入 SSA 和 MIR，使复杂优化、寄存器分配与机器级 调度能够在结构化中间表示上完成。公开 CLI 只接受比赛要求的 -S、-o 和 -O1；不公开内部研究选项。

3. 总体架构



命令行和流水线入口位于 src/main.cpp。性能配置由 src/core_ir/passes/pipeline_builder.cpp 生成，CoreIR 优化主线位于 src/core_ir/passes/passes.cpp，MIR 后端由 src/backend_mir/mir_backend.cpp 串接。

4. 源码组织

目录	职责
src/frontend/	token、lexer、parser 与 AST 定义
src/analysis/	CFG、支配、循环、函数效果、终止与访问关系分析
src/hir/	AST/HIR 规范化、高层递归与循环变换
src/core_ir/ir/	Module、Function、BasicBlock、Value、Instruction 等 SSA IR
src/core_ir/lowering/	AST 到 CoreIR 的作用域、类型、表达式和控制流降低
src/core_ir/passes/	CoreIR 优化、PassManager、流水线与目标前变换
src/rewrite/	代数、位模式、强度削弱和局部等价重写
src/backend_mir/	MIR、lowering、调度、寄存器分配、栈帧和汇编打印
src/backend/	功能通道使用的 AST 直接代码生成器
tests/riscv/	pass 语义台账和 SysY 正负夹具
tests/tools/	default/skip、trace、链接和 QEMU 门禁脚本
tools/	全量回归、汇编比较与性能分析工具

5. 前端设计

5.1 词法分析

src/frontend/lexer.cpp 将源文本转换为 token 流，识别关键字、标识符、 整数与浮点常量、运算符、分隔符和注释。token 携带源文件与位置信息，供 parser 报告错误和构造 AST 使用。词法层只负责字符到 token 的确定性转换， 不在此阶段做优化模式识别。

5.2 语法分析与 AST

src/frontend/parser.cpp 按 SysY2022 文法构造编译单元。AST 节点覆盖：

- 全局与局部常量、变量和多维数组；
- 函数定义、形式参数、函数调用与返回；
- 块、条件、while、break、continue；
- 一元、二元、关系、逻辑和赋值表达式；
- 标量与聚合初始化器。

AST 保留源程序的结构、声明和数组维度，使功能通道可以直接生成代码， 也使性能通道能在进入 SSA 前处理递归调用、数组复制和跨调用结构。

5.3 类型、作用域与语义降低

性能通道由 src/core_ir/lowering/ast_to_ir.cpp 建立符号作用域、函数签名 和类型映射。局部变量、参数、全局对象、数组下标和初始化器被转换为 CoreIR 值与内存操作；短路逻辑、条件与循环被展开为显式基本块和分支。

作用域解析以声明绑定为准，不把普通用户名称作为优化条件。main 和 SysY 运行时函数名称只用于语言及 ABI 协议。

6. 功能通道

不带 -O1 时，src/backend/codegen.cpp 从 AST 直接生成 RISC-V 汇编。 该路径负责表达式求值、局部和全局对象、控制流、函数调用、返回值以及数组 地址计算。它避免依赖复杂中端 pass，是功能正确性的独立实现路径。

功能通道与性能通道共享语言前端，因此二者对 SysY 语法和作用域的理解 一致；两条通道的后端实现相互独立，可用于发现中端或 MIR 变换引入的问题。

7. AST/HIR 高层变换

带 -O1 时，编译器在 AST 降低前执行适合高层结构的规范化和变换，包括：

- AST 常量与控制流简化；
- 尾递归消除和递归累加器规范化；
- 可证明的递归 radix 结构降低；
- 需求驱动复制和跨调用数组摘要；
- 其他只在 AST 结构仍清晰时容易验证的调用与数组变换。

高层变换必须检查调用副作用、参数使用、数组维度、读写闭包、终止条件和 结果 live-out。额外上下文参数只有在不改变 callee 语义且求值无副作用时 才能被保留或忽略。函数改名、变量改名或调用次数变化不应影响匹配结论。

AST/HIR 变换完成后，PipelineBuilder 只接收“是否保留 loop-memory 结构”、 “是否改写递归结构”等能力摘要，不根据某个 benchmark 或 pass 名称选择 后续路径。

8. CoreIR 数据模型

CoreIR 是性能通道的主要 SSA 中间表示。核心对象包括：

- Module：函数、全局变量、常量和模块级信息；
- Function：参数、返回类型、基本块和函数属性；
- BasicBlock：指令序列、前驱、后继和终结指令；
- Value：常量、参数、全局对象和指令结果的共同基类；
- Instruction：算术、比较、转换、内存、调用、Phi、分支和返回。

每个值维护 use-def 关系，基本块组成显式 CFG。Phi 指令表示控制流汇合处 的 SSA 值，GEP/地址表达式保留数组和指针计算结构。该模型支持在同一表示 上进行常量传播、死代码删除、循环分析和函数级变换。

9. 分析基础设施

AnalysisManager 统一管理函数级和模块级分析，并在 IR 变化后失效缓存。 优化常用的事实包括：

分析	用途
CFG 与支配	判断定义是否支配使用、识别回边和控制依赖
自然循环与归纳变量	建立 preheader、header、latch、exit、步长和边界
function effect	汇总全局读写、参数内存访问、调用副作用和只读函数
alias / escape	判断地址是否可能重叠、指针是否逃逸到未知调用或全局状态
termination	证明被跳过的调用和循环能够终止，并检查潜在 trap
affine / region	描述数组下标、访问区域、步长和跨迭代依赖
liveness / live-out	判断变换删除或合并的中间状态是否仍被外部观察

任何修改 CFG、Phi、调用图或内存效果的 pass 都必须重新计算或失效对应 分析。后续 pass 不能使用结构变化前的旧结论。

10. CoreIR 优化流水线

性能模式由 PipelineBuilder 启用 CoreIR 主线。优化不是单次线性列表，而是 “规范化—专项变换—清理—重新分析”的多阶段过程。

10.1 基础规范化

- 常量折叠与代数化简；
- Mem2Reg 与 Phi 构造；
- SCCP、CSE/GVN、指令合并；

- CFG 清理、不可达块和死代码删除；
- 数组 SROA、死存储和无用返回值清理。

10.2 循环与标量变换

- 循环基本化、归纳变量和不变量处理；
- 循环展开、分块、仿射边界收紧与地址递推；
- modular scalar recurrence、small linear recurrence 和重复规约；
- 运行时不变除数版本化、局部 store-load forwarding；
- loop-state fixpoint、pure-call last-value 等带终止与效果证明的变换。

重复规约只有在状态转移满足 F(acc) = acc + delta 时才能用首轮增量乘 迭代次数。旧累加器参与乘法、条件、地址或额外 live-out 时必须拒绝。

10.3 数组、调用与融合

- 函数内联与受约束的 function specialization；
- row locality、transpose view forwarding；
- producer/stage fusion、row broadcast 与 stencil interior；
- 跨调用摘要、数组递推和循环内存提升。

这类变换必须在物化新 Phi、旁路原 CFG 或删除计算前完成 alias、effect、 checksum、live-out 和路径闭包预检。预检失败时不应留下部分改写的 IR。

10.4 Rewrite 层

src/rewrite/ 处理局部、可组合的等价变换，例如：

- 常量强度削弱；
- 位模式和奇偶性规范化；
- load/store 局部化与提升；
- 数组标量替换；
- 保持有符号位宽语义的代数重写。

局部 rewrite 仍需保留整数位宽、除零、移位范围、内存顺序和调用 clobber。

11. 优化正确性原则

11.1 触发事实

默认优化只能由类型、SSA/use-def、CFG、支配、自然循环、alias、effect、 ABI 和成本模型等通用事实驱动。源文件名、输出路径、普通函数名、变量名、 固定输入、计时邻域和公开用例骨架不能决定是否变换。

11.2 内存与副作用

删除、合并或重排内存操作前，需要证明地址不别名，或证明所有可能别名的 访问顺序不变。未知调用、指针逃逸和未闭合的跨函数 effect 会使变换回退。

11.3 控制流、终止与异常行为

减少动态迭代次数或跳过调用时，不仅要比较最终数值，还要证明被跳过部分 能够终止、不会发生除零或非法移位等 trap，并且没有可观察副作用。零次 迭代、一迭代、边界步长和回绕条件需要单独处理。

11.4 成本模型

展开因子、分块规模和机器级阈值用于权衡指令数、分支、访存和寄存器压力。 这些数字必须具有目标成本含义，而不能对应某个公开输入规模。无法确认收益 时可保留原循环或选择较保守的变换。

12. strict 与 research 隔离

CMake 选项 SYC_ENABLE_RESEARCH_PASSES 默认为 OFF。strict 构建会从 编译源列表中移除尚未完成默认语义闭环的独立实现；research 构建只有在 显式设置该选项时才编译这些文件，并由统一的 research switch 控制入口。

```
# strict / 提交构建
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release \
  -DSYC_ENABLE_RESEARCH_PASSES=OFF

# 本地研究构建
cmake -S . -B build-research -DCMAKE_BUILD_TYPE=Release \
  -DSYC_ENABLE_RESEARCH_PASSES=ON
```

strict 二进制不能通过设置单个 SYC_ENABLE_* 环境变量启用 research 变换。SYC_SKIP_* 和 MIR_NO_* 用于本地 default/skip 对照与调试，不是 比赛接口，也不根据输入程序自动设置。

13. MIR 与 RISC-V 后端

lowerToMIR 将 CoreIR 的值、基本块、Phi、调用和内存操作转换为机器级 指令与虚拟寄存器。MIR 后端按以下顺序工作：

1. 初始 machine cleanup；
2. 准备 early-return shrink wrapping；
3. 对浮点热点块进行寄存器分配前调度；
4. 默认执行 linear-scan register allocation；
5. 必要时插入 spill/reload，并旋转循环分支；
6. 进行寄存器分配后的 list scheduling；
7. 依据 CFG 选择 block layout；
8. 执行分支、跳板和直落相关 peephole；
9. 清除冗余 store 及其遗留地址计算；
10. lower frame，输出最终汇编。

MIR_SIMPLE_RA=1 可在本地切换到调试用的简单分配器；默认提交路径使用 linear-scan。机器级 pass 必须维护寄存器 def/use、物理寄存器 clobber、 内存依赖和块边关系。

14. ABI、数据布局与汇编输出

目标为 RV64GC。后端负责：

- 整数与浮点参数、返回值的 ABI 寄存器分配；
- caller-saved 与 callee-saved 寄存器处理；
- 栈参数、spill slot、局部对象和栈对齐；
- i32 运算的符号扩展与 word 指令语义；
- 全局符号、字符串、整数和浮点常量段；
- 大偏移地址拆分和帧索引解析；
- .text、.data、.bss、.rodata 和必要的对齐信息。

生成的汇编通过 GNU RISC-V 工具链链接，并按比赛要求适配较大地址空间下的 medany 使用方式。输出不嵌入输入用例名称相关的优化 metadata。

15. SysY 运行时边界

比赛平台负责将生成的汇编与平台 SysY 运行时库链接。本仓库不提交自定义 运行时源码或 libsysy.a，编译器本身也不替换输入输出、计时等运行时语义。

本地需要 QEMU 回归时，可以显式提供外部运行时源码：

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release \
  -DSYC_LOCAL_RUNTIME_SOURCE=<path-to-local-sysy-runtime.c>
cmake --build build --parallel
```

只有设置 SYC_LOCAL_RUNTIME_SOURCE 时，CMake 才会在 build 目录生成本地 测试用 libsysy_runtime.a。该源码和生成库均为本地产物，不属于提交内容。

16. 构建与使用

环境要求：

- CMake 3.20 或更高版本；
- 支持 C++17 的 C++ 编译器；
- 本地链接时使用 riscv64-linux-gnu-gcc；
- 本地运行时使用 qemu-riscv64 或 qemu-riscv64-static。

构建编译器：

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --parallel
```

编译 SysY 程序：

```
./build/compiler -S -o testcase.s testcase.sy
./build/compiler -S -o testcase.s testcase.sy -O1
```

使用外部运行时进行本地链接：

```
riscv64-linux-gnu-gcc -march=rv64gc -mcmodel=medany -static \
  testcase.s <path-to-sysy-runtime.a> -lm -o testcase.elf
qemu-riscv64-static ./testcase.elf
```

17. 测试与证据体系

tests/riscv/default_pass_semantics.tsv 登记每个逻辑入口的实现、正确语义、 正例、负例、状态和证据。夹具位于 tests/riscv/，脚本位于 tests/tools/。常用验证方式包括：

- default/skip 汇编比较：确认正例确实触发、负例保持原路径；
- trace：记录接受条件或具体拒绝原因；
- 汇编、链接和 QEMU：比较输出、返回值与运行行为；
- strict/research 对照：确认研究开关不能改变 strict 汇编；
- 全量功能与性能正确性回归；
- 对有汇编变化的性能用例分析指令、分支、访存、栈帧与内部计时。

证据归档在 tests/riscv/evidence/。仓库只保存源码夹具、脚本、汇编或哈希、 QEMU 结果和汇总；不保存 build、编译器二进制、ELF 或外部运行时库。

18. 新增 pass 的接入流程

1. 明确变换前提、保持的可观察行为和需要失效的分析；
2. 在适合的 AST/HIR、CoreIR、rewrite 或 MIR 层实现；
3. 添加中文说明的独立正例和近似负例；
4. 提供 skip 与 trace，验证正例触发、负例拒绝；
5. 更新 default_pass_semantics.tsv；
6. 运行审计标准中的静态扫描和 strict/research 隔离；
7. 运行功能、性能正确性和受影响汇编分析；
8. 只有证明闭合后才进入 strict 默认路径，否则保留在 research。

项目的默认优化审计规则见 docs/compliance/generality-audit-standard.md。 该规则和测试证据共同约束后续扩展，使性能改进建立在可迁移的编译器事实 与可复核语义之上。